

Spencer Kurtz
Advisor: Fawad
Ahmad

Conclusion: Computer Vision Analysis of Volleyball Serves

Overview

Goal: Build a vision tool for volleyball serves with a fixed camera

- Detect and track the ball across frames
- Segment court lines and map boundaries
- Reconstruct serve trajectory
- Landing frame and zone
- Evaluate consistency across serves



Accomplished Work

YOLOv8s ball detector

Flight path tracker and trajectory merging

Landing frame estimation

Landing position estimation

CLI tools for processing + visualization

Data Collection and Model Training

Finished data collection

- 5 different ball types
- 4 sessions
- 6 players
- 113 serves
- 16,172 frames

Trained final model on 100 epochs using 1920x1080 images

- Previously used 50 epochs and 1280x720

Tracking and Merging

Used Simple Online Realtime Tracking (SORT)

- Fed YOLO bounding boxes
- Created trajectory fragments

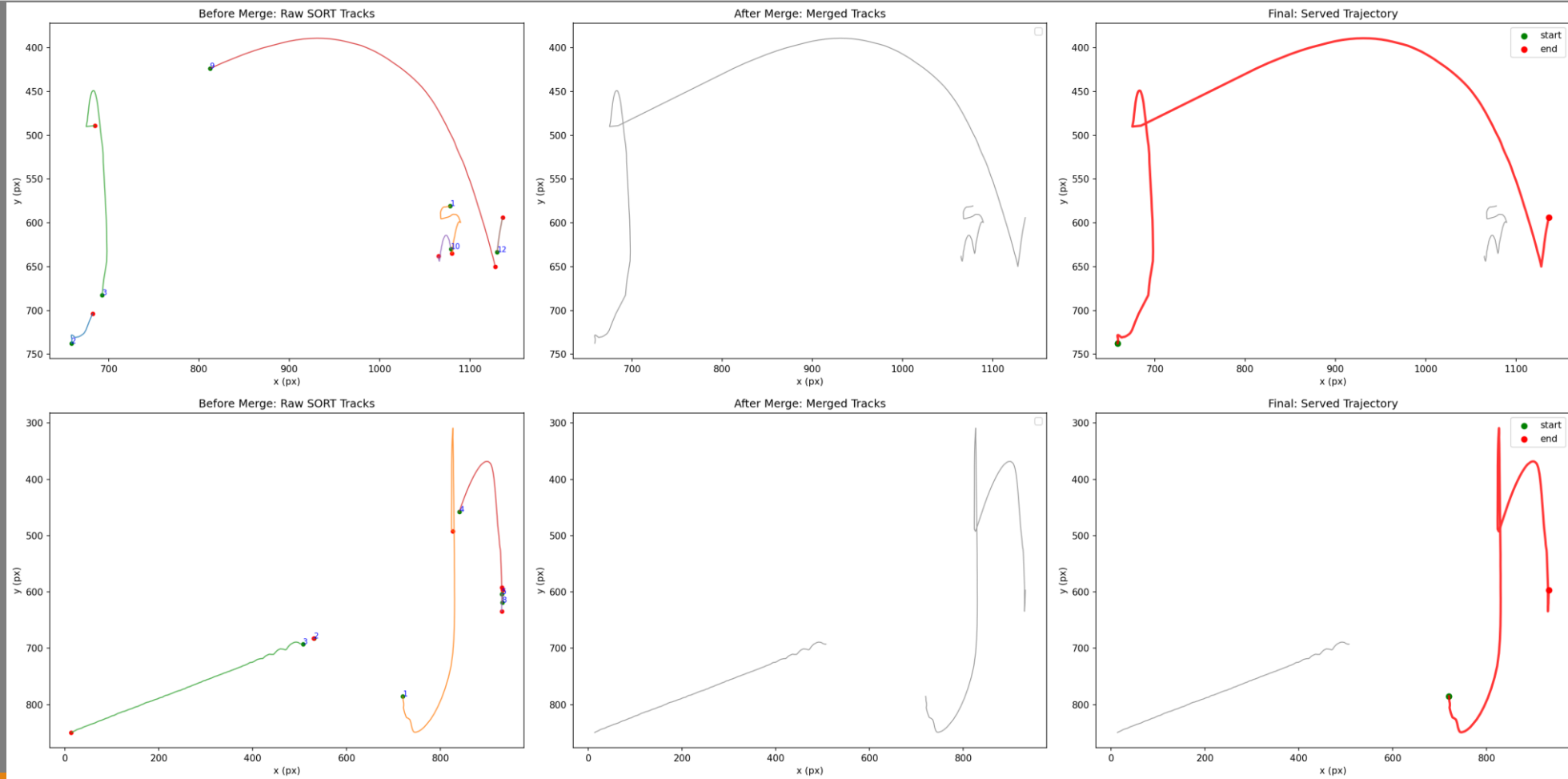
Built a greedy algorithm to combine fragments

- Compute frame gap and pixel distance between fragments
- Greedily merged into chains based on simple cost metric
 - $\alpha \cdot \text{gap} + \beta \cdot \text{dist} + 0.5 \cdot \text{overlap_penalty}$

Selected final serve based

- Chain with highest average speed
- Chain with ball size decreasing on average

Tracking and Merging



Estimate Hit and Landing Frame

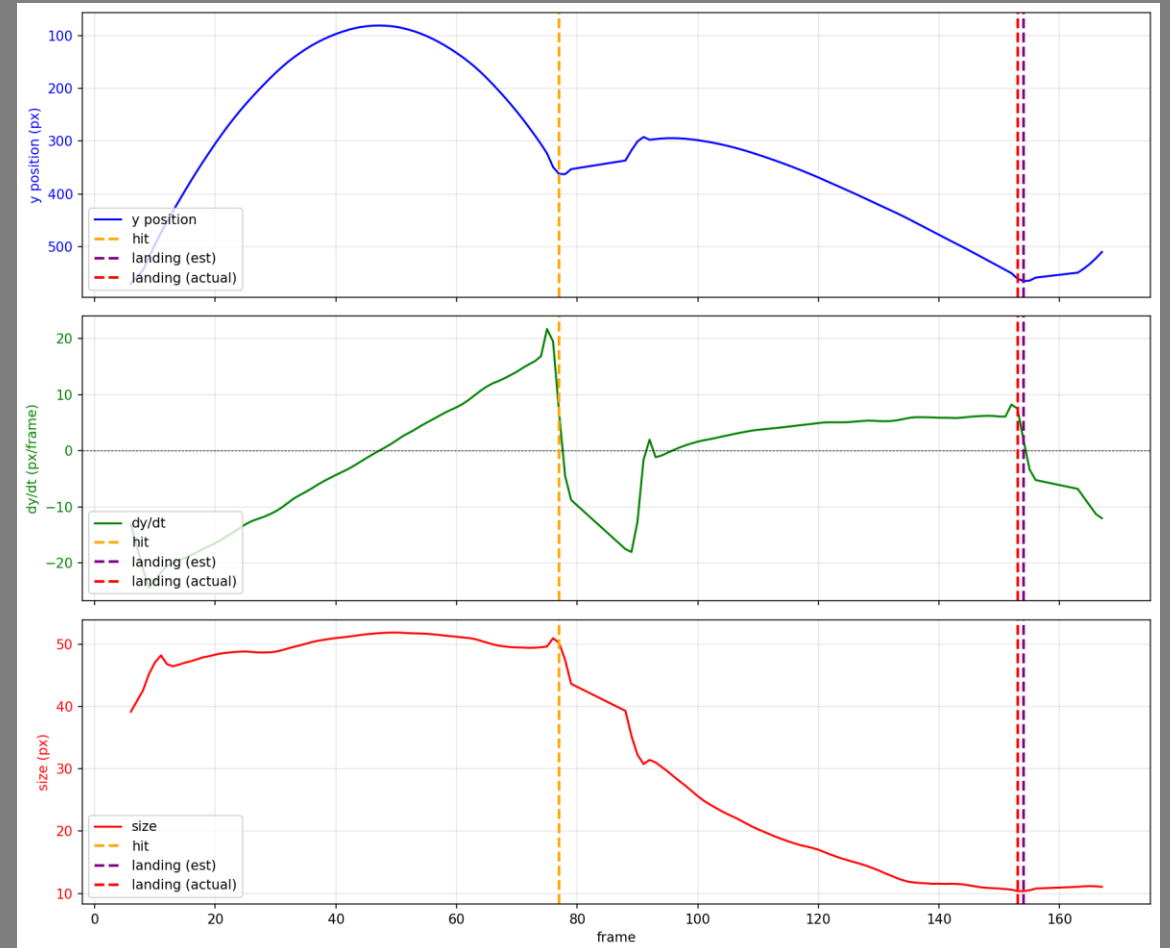
Use a Savitzky–Golay filter to smooth ball trajectory (y) and bounding box size

Detect Hit

- Compute ball y velocity - Strongest downward motion
- Use acceleration to verify - Sudden change indicates serve

Landing

- Use sliding window to compute average ball size
- Sudden velocity change and smallest ball size indicate landing



Court Homography

Use 6-point calibration to create homography map per session

Maps pixels to meters (9×18 m)

Verified length and width accuracy

Example:

```
py .\scripts\test_homography.py --session 1
```

Width near baseline: 9.06 m (expected: 9.0 m)
Width far baseline: 8.92 m (expected: 9.0 m)
Length left side: 18.00 m (expected: 18.0 m)
Length right side: 18.00 m (expected: 18.0 m)



Landing Position Calculation

Homography lets us map pixels to meters

Example:

```
py .\scripts\project_trajectory.py
```

```
--player spencer -- session 3 -- serve 3
```

Projected landing 3 → (6.65m, 14.81m)



Current Conclusions

Single-camera setup works reliably

YOLOv8 + SORT achieve stable tracking

Landing frame accurate within a few frames

Calibration matches real-court scale

Will work well after parameter calibration

Next Steps

Serve

- Use MediaPipe to detect a hand signal indicating the start of a serve

Analysis

- Create heatmaps based on landing position

Serve Characteristics

- Average speed
- Average flight duration
- Average peak height

Finish user CLI

Future Work

Expand to support multiple camera angles

Create automatic line detection

GUI for user

Decrease current runtime

- Optimize YOLO ball tracking



Questions

